Write-up: Replace

por *dleskow*

Informações do chall

- **Título**: Reversing.Kr/Replace
- **Autor**: —
- **Contexto**: —
- **Arquivos**:
  - *Replace.zip*
    - *Replace.exe*

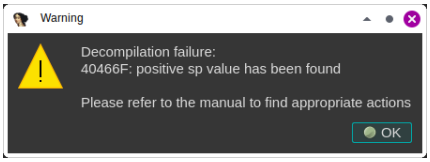
1. Write-Up

1.1. Análise Estática

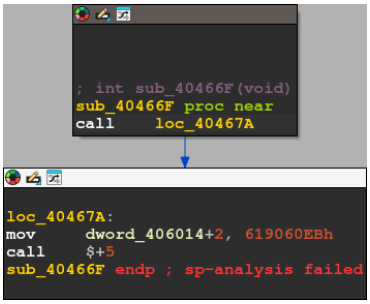
Abrindo no IDA, temos a seguinte função principal (já com spoilers nos comentários):

```
INT_PTR __stdcall DialogFunc(HWND hDlg, UINT a2, WPARAM a3, LPARAM a4)
{
  if ( a2 != 0x111 )
    return 0;
  if ( (unsigned __int16)a3 == 2 )
  {
    EndDialog(hDlg, 2);
    return 1;
  }
  else if ( (unsigned __int16)a3 == 1003 )
  {
    dword_4084D0 = GetDlgItemInt(hDlg, 1002, 0, 0); /* Lê long long int para 0x4084d0 */
    sub_40466F();                                   /* Função não analisável, mas muito usada */
    ((void (__cdecl *)(void *))sub_404689)(&loc_40469F);
    *(_DWORD *)sub_40466F = 0xC39000C6;             /* opcode: muda o próprio código durante execução */
    sub_40466F();
    sub_40466F();
    *(_DWORD *)sub_40466F = 0x6E8;                  /* De novo */
    return 1;
  }
  else
  {
    return 0;
  }
}
```

Se formos ver quem é esta função que é chamada três vezes, `sub_40466F`, recebemos a seguinte mensagem de erro do IDA:



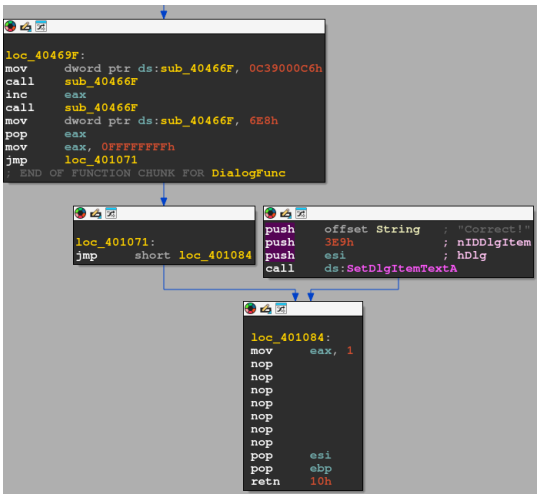
E, tentando ver a função pela visualização em grafos, temos:



Ou seja, não conseguimos tirar nada útil desta função pelo IDA. O motivo disto é que, como pus nos comentários da `DialogFunc` decompilada, o programa altera o proprio código durante execução o que impossibilita uma análise estática.

Dito isso, o resto da solução utiliza análise dinâmica.

Além disso, podemos ver também pela visualização de grafo do IDA onde queremos chegar, que é a região que usa a string `"Correct!"`, sendo que as intruções que a usam estão localizadas no endereço `0x401073` (basta clicar na instrução e ver o endereço na parte de baixo, ou então botão direito e mandar sincronizar a aba com uma aba *Hex View* para ver o endereço). Apesar de isto nem ser uma função direito, chamarei nossa região de interesse de função win():



1.2. Análise Dinâmica

Por motivos de facilidade, como o chall de Windows foi resolvido no Linux, utilizou-se o GDB, juntamente ao Wine, através do seguinte comando:

```
┌──(dleskow㉿ganesh)-[/home/ganesh/Reversing.Kr/Replace]
└─# winedbg --gdb Replace.exe
```

Nota-se que, colocando um valor qualquer, o programa sempre crash com o seguinte *seg fault*:

```
-------------------------[ DISASM / i386 / set emulate on ]-------------------------
► 0x40466f  mov    byte ptr [eax], 0x90        <Cannot dereference [0x601606cb]>
```

Onde o valor é sempre

```
eax = 0x601605cb + input_value
```

Para entendermos melhor o que está acontecendo, vejamos a execução do código linha a linha. Primeiro colocamos o *breakpoint* de acordo com o que queremos (e sabemos pelo IDA); recomendo também estender o número das linhas no *context*, para facilitar a visualização:

```
pwndbg> b* 0x401065
Breakpoint 1 at 0x401065
pwndbg> set context-disasm-lines 50
Set number of additional lines to print in the disasm context to 50.
```

Dois endereços para se manter em mente daqui para a frente são:

```
0x4084d0 # Endereço do seu input
0x401073 # Função win(), visto pelo IDA (é quem printa o "Correct!")
```

Executando o programa a partir de `0x401065` e coletando as linhas executadas, aqui está o que está acontecendo (para clareza, foi dada ênfase (►) onde há manipulação do valor de entrada e foram adicionados comentários explicitando o quão profundo se está transicionando nas funções (0 é o nível inicial)):

```
------------------------[ DISASM / i386 / set emulate on ]------------------------
  0x401065  call   0x40466f        <0x40466f>  // 0 -> 1

  0x40466f  call   0x40467a        <0x40467a>  // 1 -> 2

  0x40467a  mov    dword ptr [0x406016], 0x619060eb        [0x406016] <= 0x619060eb
  0x404684  call   0x404689        // 2 -> 3

► 0x404689  inc    dword ptr [0x4084d0]
  0x40468f  ret            <0x404689>  // 3 -> 2
    ↓
► 0x404689  inc    dword ptr [0x4084d0]
  0x40468f  ret            <0x404674>  // 2 -> 1
    ↓
► 0x404674  add    dword ptr [0x4084d0], 0x601605c7        // Eis o princípio de nossa constante somada à entrada
  0x40467e  inc    eax        EAX => 0x101
  0x40467f  add    bl, ch         BL => 73 (0x4a + 0xff)
  0x404681  pushal
  0x404682  nop
  0x404683  popal
  0x404684  call   0x404689        // 1 -> 2

► 0x404689  inc    dword ptr [0x4084d0]
  0x40468f  ret            <0x404689>  // 2 -> 1
    ↓
► 0x404689  inc    dword ptr [0x4084d0]
  0x40468f  ret            <0x40106a>  // 1 -> 0
    ↓
  0x40106a  xor    eax, eax        EAX => 0
  0x40106c  jmp    0x404690        <0x404690>
    ↓
► 0x404690  mov    eax, dword ptr [0x4084d0]        EAX, [0x4084d0]  // Passa nossa entrada para eax
  0x404695  push   0x40469f
  0x40469a  call   0x404689        <0x404689>  // 0 -> 1

  0x404689  inc    dword ptr [0x4084d0]
  0x40468f  ret            <0x40469f>  // 1 -> 0
    ↓
  0x40469f  mov    dword ptr [0x40466f], 0xc39000c6        [0x40466f] <= 0xc39000c6  // Muda o código
  0x4046a9  call   0x40466f        <0x40466f> // 0 -> 1

► 0x40466f  mov    byte ptr [eax], 0x90        <Cannot dereference [0x601606cb]>  // Muda um byte para 0x90 = nop
```

Veja que ele crasha tentando escrever `0x90` em um endereço que controlamos e, coincidentemente, `0x90` é o *opcode* para `nop`.

Passemos um endereço arbitrário para ver o que ele faz após isto. Para controlar o endereço, basta utilizar a equação de antes e dar overflow no valor (`byte ptr` em 32-bits, então utiliza apenas os últimos 4 bytes).

Por exemplo, para passar o endereço `0x40108b`, fazemos:

```
0x10040108b - 0x601605cb = 0xa02a0ac0 = 2687109824
```

e enviamos este valor em decimal para o programa.

Passando este valor, vemos o seguinte código sendo executado:

```
-------------------------[ DISASM / i386 / set emulate on ]-------------------------
► 0x40466f  mov    byte ptr [eax], 0x90        [0x40108b] <= 0x90
  0x404672  ret             <0x4046ae>  // 1 -> 0
    ↓
► 0x4046ae  inc    eax         EAX => 0x40108c
  0x4046af  call   0x40466f         <0x40466f>  // 0 -> 1

► 0x40466f  mov    byte ptr [eax], 0x90        [0x40108c] <= 0x90  // Muda também o próximo byte para 0x90 = nop
  0x404672  ret             <0x4046b4>
    ↓
  0x4046b4  mov    dword ptr [0x40466f], 0x6e8        [0x40466f] <= 0x6e8
  0x4046be  pop    eax         EAX => 0x40469f
  0x4046bf  mov    eax, 0xffffffff         EAX => 0xffffffff
  0x4046c4  jmp    0x401071         <0x401071>  // Pula para perto da win()
    ↓
  0x401071  jmp    0x401084         <0x401084>
    ↓
  0x401084  mov    eax, 1         EAX => 1
  0x401089  nop
  0x40108a  nop
  0x40108b  nop
  0x40108c  nop
  0x40108d  nop
  0x40108e  nop
  0x40108f  nop
  0x401090  pop    esi         ESI => 0x3eb
  0x401091  pop    ebp         EBP => 0x50f30c
  0x401092  ret    0x10         <WINPROC_wrapper+28>
```

Ou seja, somos capazes de sobrescrever dois bytes consecutivos com `0x90`.

Veja só que coincidência: depois de sobrescrever **dois** bytes com `nop`, o programa pula para `0x401071`, que é **dois** bytes antes de nossa função win(), em `0x401073`!

Dito isso, basta fazer as mesmas contas de antes, mas passando o endereço `0x401071`, de forma a sobrescrever `0x401071` e `0x401072` com `nop` e, desta forma, alterar o fluxo de excução para nossa win()!

```
0x100401071 - 0x601605cb = 0xa02a0aa6 = 2687109798 # Resposta
```